IEICE Electronics Express, Vol.20, No.11, 1–6

LETTER

QIAD: A quadratic interpolation approximate divider

Hao Liu[1, a)], **Ming-Jiang Wang**[1], **and Ming Liu**[2, b)]

Abstract The balance between computational performance and power consumption is a key constraint in computing systems, with integrated circuit technology posing a bottleneck. Approximate computation can trade off accuracy for performance or power improvement in error-tolerant scenarios. Division, with high computational demands and latency, is a bottleneck for computational efficiency. We propose a quadratic interpolation approximate divider (QIAD) based on multiplicative division, which has superior statistical performance. The design is simulated and synthesized at TSMC 65 nm process and tested on image color quantization, showing the best quantization effect using evaluation indicators such as PSNR, MSE, and SSIM.

Keywords: approximate computing, divider, hardware design
Classification: Integrated circuit (logic)

1. Introduction

Energy efficiency is a formidable challenge in computing system design, and these requirements for most computational devices are accompanied by a demand for low power/energy consumption. And energy-efficient arithmetic units are a key building block to achieve this goal [1, 2]. Researchers have been seeking to develop fast and low-power arithmetic algorithms and circuit implementations, and as a result, various designs have been proposed to improve the performance of arithmetic operations and reduce power consumption. In the traditional computing paradigm, arithmetic operations are expected to be always accurate, so arithmetic units are designed to produce strictly accurate results at all times. However, some applications are more forgiving and do not require such a high precision level depending on the environment [3]. For such applications, using always-accurate arithmetic units is a waste of energy without any quality improvement. Many applications, such as multimedia and image processing, can tolerate errors and inaccuracies in computation and still produce meaningful and useful results. Approximate computing is an emerging computational paradigm that accomplishes energy-efficient computation by relaxing the accuracy requirements for applications that do not always require exact computational results [4]. This method will redirect existing design processes by exploiting reductions in complexity and cost, and potentially improving performance and power efficiency.

Arithmetic blocks consume a large fraction of the total power consumption of a digital processing system, and their speed has a significant impact on the overall system performance. Among the four basic arithmetic operations, while the division operation is executed less frequently, its latency and energy consumption are larger than the other operations. In application scenarios with high computational demands division computation becomes a bottleneck in computational efficiency, and improving the computational efficiency of division can significantly improve the computational efficiency of the whole system.

The rest of the paper is organized as follows: Section 2 discusses the related work and summarizes the existing approximate dividers. Quadratic interpolation approximate divider is proposed and the circuit structure of the approximate divider will be introduced in Section 3. The performance evaluation and error analysis of the proposed approximate divider will be presented in Section 4. Also application tests are performed to verify the usability of the approximate divider. Finally, conclusions are presented in Section 5.

2. Previous work

For the present stage, a large number of divider designs have been proposed by many scholars [5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21]. This section will give a brief introduction to the common approximate divider designs.

An energy-efficient, high-performance divider [22] is proposed based on logarithmic conversion and piecewise linear approximation. A heuristic search algorithm is then designed to find the most accurate set of constants to approximate the reciprocal of the divisor by minimizing statistical errors. This paper [23] presents an approximate hybrid divider (AXHD). It exploits the advantages of both logarithmic divider (LD) and exact array divider (EXD) to achieve a trade-off between hardware performance and accuracy. The accurate recovery division unit generates the most significant bits (MSBs) of the quotient to achieve high accuracy. In contrast, the other quotient bits are generated using LD as an approximate scheme to improve power consumption, area, and latency advantages. This paper [24] presents an energy-efficient and logarithm-based approximate divider (LEAD) for ASIC and FPGA-based applications. The LEAD is based on a functional approximation of an arithmetic divider by rounding the input to the nearest power of two and calculating the division using basic addition/subtraction operations and shifters. The proposed approximation design is error configurable and can be used for both signed and unsigned

[1] Faculty of Electronics and Information Engineering, Harbin Institute of Technology, Shenzhen 518055, GuangDong, China
[2] Sino-German School, Shenzhen Institute of Information Technology, Shenzhen 518055, GuangDong, China
[a)] gabriel.carson.law@outlook.com
[b)] lm_hit_1986@126.com

DOI: 10.1587/elex.20.20230167
Received April 12, 2023
Accepted April 18, 2023
Publicized May 2, 2023
Copyedited June 10, 2023

integers. In this paper [25], researchers presented an approximate model for recovering a divider. To improve the accuracy over other existing approximate dividers, they designed the divider unit to produce a truth table similar to the accurate divider. An approximate divider [26] for dynamic energy mass scaling is proposed that involves a trade-off between accuracy and delay. The proposed divider, called the accuracy scalable approximate divider based on restoring division (ASAD-RD), uses restoring division to improve the error of the approximate divider significantly and has a much smaller computational delay.

Researchers present a new method [27] for inverse and iterative approximation of integer data using the Newton-Raphson method, designed to achieve results close to exact values and enable efficient performance. This paper [28] presents the reconfigurable energy-efficient approximate divider (READ), which implements several configurable modes of energy quality using a fixed recovery array divider architecture. The READ achieves energy efficiency while meeting the dynamically changing accuracy requirements of the target application. The READ uses reconfigurable subtractor cells that can operate in either exact or approximate modes using subtractor cell controller logic. This work [29] presents dedicated hardware architectures based on normalized minimum-mean adaptive filtering algorithms for power line harmonic interference cancellation. These hardware architectures are based on a 2's complement representation, described in VHDL, and synthesized into a 65 nm CMOS dedicated ASIC. This paper [30] proposes a reconfigurable approximate recovery of dividers and square roots based on bit weight to improve energy efficiency. Configurable subtractor cells (CSCs) are proposed, which work accurately and approximately, along with simplified and scalable overflow detection hardware as the main design unit. The proposed approximate design can switch between approximate and exact modes of operation, making it suitable for error-tolerant and error-sensitive applications.

3. Proposed design and hardware structure

3.1 Quadratic curve fitting

Traditional dividers are mostly based on iterative algorithm which computing speed is relative low, while the other design idea is multiplicative division, which has a great improvement in computation speed but requires additional cost consumption. In this section, we present our divider QIAD (Quadratic Interpolation Approximate Divider) and show the corresponding hardware implementation.

The proposed design QIAD, uses a multiplicative division algorithm. In multiplicative division, we first obtain the reciprocal of the divisor and then multiply the obtained reciprocal by the divisor to get the final result.

We use A to represent the dividend and B to represent the number of divisor. We can regularize A and B by expressing them as the following equation.

$$A = 2^{e_a} \times a, B = 2^{e_b} \times b \tag{1}$$

In which $0.5 \leq a < 1, 0.5 \leq b < 1$, where e_b and e_a stand for the corresponding exponent when the binary numbers are represented, so the quotient value Q is:

$$Q = \frac{A}{B} = 2^{e_a - e_b} \times \frac{a}{b} = 2^{e_a - e_b} \times a \times R(b) \tag{2}$$

where $R(b)$ is the reciprocal of b, in the multiplicative division, we do not calculate a/b directly. We will first calculate the reciprocal of b, then $R(b)$ multiply a by the reciprocal result and perform a shift operation to obtain the final division result.

For example, Assuming the dividend is 195 and the divisor is 12, according to the multiplication division algorithm, we will get:

$$A = 195 = 2^8 \times 0.76171875$$
$$B = 12 = 2^4 \times 0.75$$

Therefore, we will get the the quotient

$$Q = 2^{8-4} \times 0.76171875 \times (\frac{1}{0.75})$$
$$= 16.25$$

By computing the reciprocal of 0.75, the division process can be converted into shift and multiply calculations. This method can significantly improve the speed of division calculation as multiplication calculation is faster than traditional iterative division. However, the computational cost of reciprocal calculation is high, and in approximate algorithms, the computational cost of reciprocal operation can be reduced by approximating the solution.

So the key to approximate division is quickly finding the corresponding reciprocal value $R(b)$. In approximate computation, we do not need to obtain full computational precision, so we can sacrifice computational accuracy to obtain the approximate reciprocal quickly. A common means of approximating the reciprocal operation is to use the Taylor series. However, as the number of series terms increases, the computational cost required increases significantly. We use a quadratic curve fitting method in this design to obtain approximate reciprocal calculations. The same computational performance can be obtained at a smaller computational cost than Taylor series expansion.

We can construct the cost function:

$$F(a,b,c) = \int_m^n (ax^2 + bx + c - f(x))^2 \, dx \tag{3}$$

Derive for a, b, c respectively to solve the best fitting function, we can get the following equation:

$$\begin{aligned}
\frac{\partial F}{\partial a} &= -2\int_m^n x^2 f(x)\,dx + \frac{2}{5}a\left(n^5 - m^5\right) + \frac{1}{2}b\left(n^4 - m^4\right) + \frac{2}{3}c(n^3 - m^3) = 0 \\
\frac{\partial F}{\partial b} &= -2\int_m^n x f(x)\,dx + \frac{1}{2}a\left(n^4 - m^4\right) + \frac{2}{3}b\left(n^3 - m^3\right) + c(n^2 - m^2) = 0 \\
\frac{\partial F}{\partial c} &= -2\int_m^n f(x)\,dx + \frac{2}{3}a\left(n^3 - m^3\right) + b\left(n^2 - m^2\right) + 2c(n - m) = 0
\end{aligned} \tag{4}$$

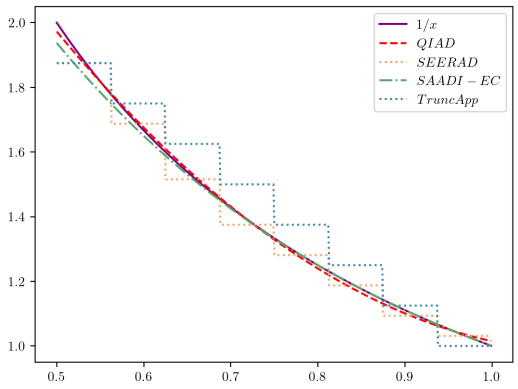
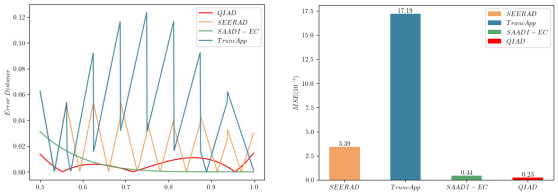


Fig. 1 Approximate reciprocal fitting comparison.



(a) RED comparison (b) MSE comparison

Fig. 2 Fitting error comparison.

Substitute the corresponding parameters, where $m = 0.5$, $n = 1$, $f(x) = \frac{1}{x}$. Finally we solve the equations and get the corresponding fitted quadratic function results as follows:

$$R(y) = 2.64x^2 - 5.875x + 4.25 \tag{5}$$

Various approximate multiplicative division methods have proposed different approaches for approximating the reciprocal. The following figure compares the fitting performance of different approximate reciprocal methods, with the proposed algorithm represented as QIAD. Figure 1 shows the fitting results of several similar designs. The reciprocal fitting in SEERAD and TruncApp algorithms shows a stepwise pattern because only the high-weighted bit positions of the data are utilized. On the other hand, QIAD and SAADI-EC use polynomial approximation, resulting in smoother fitting curves.

Figure 2(a) presents the relative error of the fitting performance of several methods. It can be seen from the figure that the TruncApp algorithm has the largest relative error, while the proposed QIAD algorithm has the most evenly distributed error. Figure 2(b) shows the fitting performance of several algorithms in terms of mean squared error (MSE). The proposed QIAD algorithm clearly has the smallest MSE, indicating the best fitting performance among the methods compared.

3.2 Hardware structure

In this section, we present the hardware architecture design of QIAD. The computational flow of QIAD can be divided into the following steps.

1. The division is checked and regularized using the leading zero detection detector circuit. A shift operation is performed so that the 1 in the highest bit of the data becomes the highest bit, and the shift count k is recorded.
2. The regularized data is fed into the approximate reciprocal calculation module to obtain the corresponding approximate calculation result.
3. Multiply the reciprocal result with the dividend and shift the corresponding shift count to the right to obtain the final division result.

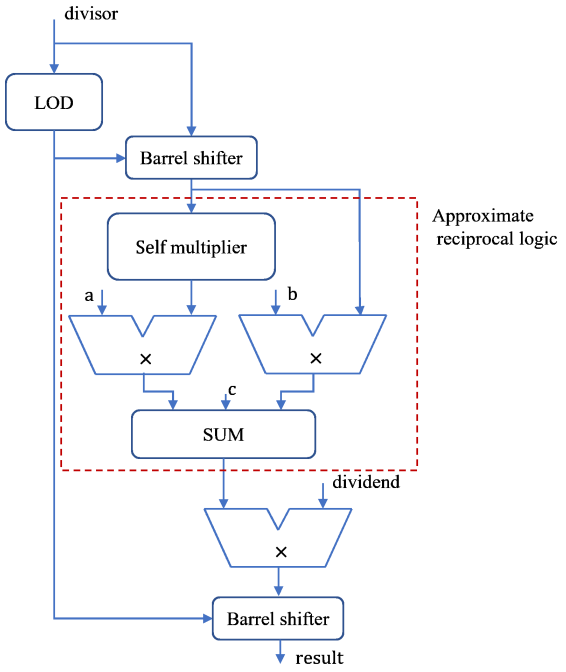


Fig. 3 Top-level structure of QIAD.

Figure 3 shows the hardware structure of the proposed approximate divider in the paper. In the hardware implementation, we did not normalize the dividend, but instead kept its original value and only processed the divisor. First, the divisor is input to the leading zero detection module, and it is normalized based on the result of the leading zero detection circuit with the shifter to obtain the corresponding b in the algorithm mentioned earlier. The structure within the red dashed box corresponds to the process of approximate reciprocal calculation in the algorithm, where the input is the normalized b and the output is the approximate reciprocal result. This is also the most computationally expensive part of the QIAD algorithm. It includes a self-multiplier for calculating the square term, followed by two multiplications for coefficient multiplication. Finally, the approximate reciprocal result $R(b)$ is obtained through addition and merge logic. Then, the result is multiplied with the dividend and further processed with the shifter to obtain the final division result.

4. Result and analysis

4.1 Error analysis

To confirm the error performance of the proposed QIAD, we compared the performance of the QIAD with several comparable state-of-the-art approximate dividers. These divisors include SEERAD, SADDI-EC, TruncApp, etc. We have implemented 8-bit,16bit, and 32-bit versions in Verilog language. We used Python language to model and tested each design at the algorithm level. For the 8-bit design, we generate full coverage test sets, and for the 16-bit

Table I ER comparation with RED restriction.

RED (%)	≤ 1%	≤ 2%	≤ 5%	≤ 10%	≤ 15%
TruncApp	7.45%	25.10%	58.43%	94.51%	100%
SEERAD	24.31%	45.10%	90.20%	100%	100%
SAADI-EC	24.71%	100%	100%	100%	100%
QIAD	81.57%	100%	100%	100%	100%

Table II Error statistic.

	MRED (%)	MED (%)	NMED (%)	PE (%)
TruncApp	4.59	13.87	0.87	11.91
SEERAD	2.42	12.52	0.79	6.52
SAADI-EC	1.23	4.04	0.98	1.61
QIAD	0.60	2.48	0.71	1.38

and 32-bit data bit widths, we randomly generate 100,000 sets of test data. In this section, we discuss and analyze the error performance. We will evaluate the proposed QIAD divider in terms of MRED, MED, NMED, AR, etc., where MRED stands for mean relative error distance, MED stands for mean error distance and NMED means normalized mean error distance.

Error rate (ER) is a very common metric in approximate arithmetic unit design and is usually combined with different metrics to form an acceptability metric for the performance of the design. Table I shows the comparison of acceptability when using RED as the boundary condition.

We use RED as the boundary condition, and since the relative error of the design is small, we divide the interval more carefully in a small range and compare the acceptability of the four designs. From the table, we can see that TruncApp has the worst AR performance with a relative error distribution within 10%. The proposed QIAD has the best AR performance. The relative errors are mainly distributed around 1%.

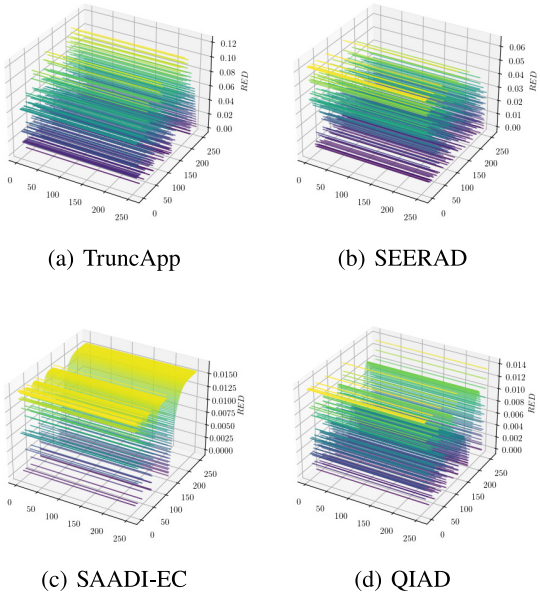
MED is a common error indicator in approximate calculation applications, and it represents the mean error distance. MED can be significantly affected by the size of the data. Therefore, normalized mean error distance (NMED) and mean relative error distance (MRED) are better error metrics. These two metrics are also the main evaluation indicators in this paper.

If we use D to represent the maximum error, then NMED can be expressed as

$$\mathrm{NMED} = \frac{\mathrm{MED}}{D} = \frac{1}{2^{2n}} \sum_{i=1}^{2^{2n}} \frac{\mathrm{ED_i}}{D} \tag{6}$$

Table II shows the various error performances of SAADI-EC, SEERAD, TruncApp, and the proposed QIAD structure. It includes NMED, MRED, NMED and PE (peak error). The table clearly shows that the proposed QIAD has the best performance for each error performance. SAADI-EC has better MRED and MED performance but has the worst NMED. Besides, TruncApp has the worst error performance in all metrics.

Figure 4 shows the relative error distributions of four dividers, SAADI-EC, SEERAD, TruncApp, and QIAD. According to the figure, it can be seen that SAADI-EC and QIAD have similar error distributions. However, QIAD has the sparsest error distribution, the smallest error distribution range, and the best relative error distribution.



(a) TruncApp (b) SEERAD

(c) SAADI-EC (d) QIAD

Fig. 4 The RED distribution.

Table III Hardware comparison.

Method	D	Area (μm^2)	Power (μW)	Delay (ns)
TruncApp	8	487.2	0.18	2.0
	16	1005.2	0.31	3.74
	32	2053.6	0.62	6.03
SEERAD	8	770.4	0.16	2.46
	16	1510.4	0.38	4.20
	32	2970.0	0.745	7.13
SAADI-EC	8	2674.4	0.51	7.04
	16	9989.2	3.85	12.24
	32	23052.0	16.22	21.62
QIAD	8	1624.8	0.33	4.32
	16	4969.2	1.69	6.93
	32	16402.4	7.66	12.01

4.2 Circuit consumption

We simulated and synthesized the proposed design at TSMC 65 nm process. The area power consumption and latency were evaluated in three cases, 8 bit, 16 bit, and 32 bit, and the results are shown in Table III.

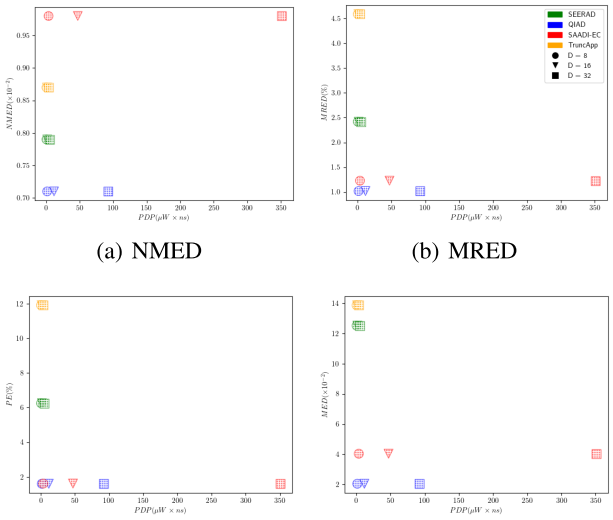
Figure 5 shows the performance of several dividers with the PDP as the horizontal coordinate and the performance parameters as the vertical coordinates. The dots in the figure represent the corresponding divider designs for different parameters. Our proposed QIAD performs well for all error metrics and is a suitable PDP sacrifice for short-bit width data.

To validate the use of QIAD in practical applications. We tested the divider design with the color quantization using k-means clustering. Several images commonly used in image processing were used as test data for the tests. The results are presented in Table IV.

Figure 6 shows the color quantization results of the proposed QIAD with Lena as an example, and the results show that our proposed design can perform the image quantization task excellently and with the highest average PSNR and

Table IV Color quantization comparison.

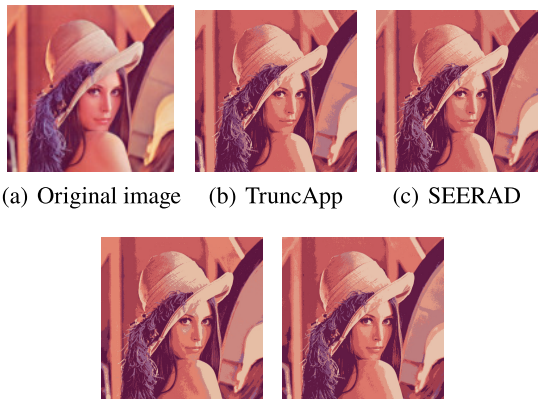
Figure	TruncApp			SEERAD			SAADI-EC			QIAD		
	MSE	PSNR	SSIM	MSE	PSNR	SSIM	MSE	PSNR	SSIM	MSE	PSNR	SSIM
peppers	54.51	29.16	0.73	33.71	31.13	0.75	23.06	33.09	0.85	18.12	34.06	0.88
baboon	80.02	28.27	0.74	64.67	29.24	0.79	60.01	29.56	0.84	27.62	32.93	0.91
couple	58.22	25.47	0.79	45.46	25.57	0.83	29.28	27.05	0.87	17.32	29.61	0.90
airplane	44.51	30.35	0.77	47.18	30.18	0.80	35.44	31.34	0.82	14.84	35.12	0.87
lena	67.77	28.92	0.68	53.33	29.96	0.73	42.24	30.97	0.79	41.55	31.20	0.82
house	90.92	27.25	0.72	57.64	29.35	0.75	46.23	29.87	0.82	21.80	33.10	0.91
villa	47.06	30.42	0.82	48.17	30.09	0.82	26.88	32.82	0.89	19.09	34.34	0.93
average	63.29	28.55	0.75	50.02	29.36	0.78	37.59	30.67	0.84	22.91	32.91	0.89



(a) NMED (b) MRED

(c) PE (d) MED

Fig. 5 Performance and consumption trade-off.



(a) Original image (b) TruncApp (c) SEERAD

(d) SAADI-EC (e) QIAD

Fig. 6 Color quantization result comparsion.

SSIM performance.

5. Conclusion

This paper proposed an approximate divider based on quadratic curve fitting method. Compared with dividers of similar design ideas, the proposed design has the best statistical performance. We have performed the synthesis at TSMC 65 nm process, and the results show that the proposed design has a moderate area and power consumption. Finally, the application test of the QIAD was carried out using k-means clustering color quantization, and the results show that the proposed design has the best test performance.

Acknowledgments

This work was supported by the project of Shenzhen Science and Technology innovation committee (JCYJ20180307123857045) and Basic Research Discipline Layout Project of Shenzhen under Grant 2020B1515120004, Grant JCYJ20180507182241622, and Grant JCYJ20180503182125190.

References

[1] H. Jiang, *et al.*: "Approximate arithmetic circuits: A survey, characterization, and recent applications," Proc. IEEE **108** (2020) 2108 (DOI: 10.1109/JPROC.2020.3006451).
[2] K. Chen, *et al.*: "A survey of approximate arithmetic circuits and blocks," it-Information Technology **64** (2022) 79 (DOI: 10.1515/itit-2021-0055).
[3] H. Jiang, *et al.*: "Approximate arithmetic circuits: A survey, characterization, and recent applications," Proc. IEEE **108** (2020) 2108 (DOI: 10.1109/JPROC.2020.3006451).
[4] J. Han and M. Orshansky: "Approximate computing: an emerging paradigm for energy efficient design," ETS (2013) 1 (DOI: 10.1109/ETS.2013.6569370).
[5] H. Jiang, *et al.*: "Adaptive approximation in arithmetic circuits: a low-power unsigned divider design," DATE (2018) 1411 (DOI: 10.23919/DATE.2018.8342233).
[6] K.M. Reddy, *et al.*: "Design of approximate dividers for error tolerant applications," MWSCAS (2018) 496 (DOI: 10.1109/MWSCAS.2018.8623909).
[7] I. Syafalni, *et al.*: "A novel approximate divider using binomial expansion," TSSA (2020) 1 (DOI: 10.1109/TSSA51342.2020.9310815).
[8] M.H. Takaby and S.M. Sayedi: "Low power approximate unsigned divider design using gate diffusion input logic," ICEE (2019) 66 (DOI: 10.1109/IranianCEE.2019.8786452).
[9] M. Vaeztourshizi, *et al.*: "An energy-efficient, yet highly-accurate, approximate non-iterative divider," ISLPED (2018) 1 (DOI: 10.1145/3218603.3218650).
[10] L. Chen, *et al.*: "On the design of approximate restoring dividers for error-tolerant applications," IEEE Trans. Comput. **65** (2015) 2522 (DOI: 10.1109/TC.2015.2494005).
[11] H. Jiang, *et al.*: "Low-power unsigned divider and square root circuit designs using adaptive approximation," IEEE Trans. Comput. **68** (2019) 1635 (DOI: 10.1109/TC.2019.2916817).
[12] L. Chen, *et al.*: "Design, evaluation and application of approximate high-radix dividers," IEEE Trans. Multi-Scale Comput. Syst. **4** (2018) 299 (DOI: 10.1109/TMSCS.2018.2817608).
[13] K.V. Krishnan, *et al.*: "Design of energy efficient approximate subtractors and restoring dividers for error tolerant applications," Microelectronics J. **131** (2023) 105668 (DOI: 10.1016/j.mejo.2022.105668).
[14] A. Gorantla and P. Deepa: "Design of approximate subtractors and dividers for error tolerant image processing applications," J. Electron

Test. **35** (2019) 901 (DOI: 10.1007/s10836-019-05837-5).
[15] P. Gowtham, *et al.*: "Design of approximate restoring dividers for error resilient applications," ECS Trans. **107** (2022) 13675 (DOI: 10.1149/10701.13675ecst).
[16] E. Adams, *et al.*: "Approximate restoring dividers using inexact cells and estimation from partial remainders," IEEE Trans. Comput. **69** (2019) 468 (DOI: 10.1109/TC.2019.2953751)
[17] M. Imani, *et al.*: "CADE: configurable approximate divider for energy efficiency," DATE (2019) 586 (DOI: 10.23919/DATE.2019.8715112).
[18] S. Vahdat, *et al.*: "TruncApp: a truncation-based approximate divider for energy efficient DSP applications," DATE (2017) 1635 (DOI: 10.23919/DATE.2017.7927254).
[19] R. Zendegani, *et al.*: "SEERAD: a high speed yet energy-efficient rounding-based approximate divider," DATE (2016) 1481 (DOI: 10.3850/9783981537079_0521).
[20] S. Behroozi, *et al.*: "SAADI: a scalable accuracy approximate divider for dynamic energy-quality scaling," ASPDAC (2019) 481 (DOI: 10.1145/3287624.3287668).
[21] J. Melchert, *et al.*: "SAADI-EC: a quality-configurable approximate divider for energy efficiency," IEEE Trans. Very Large Scale Integr. (VLSI) Syst. **27** (2019) 2680 (DOI: 10.1109/TVLSI.2019.2926083).
[22] Y. Wu, *et al.*: "Energy-efficient approximate divider based on logarithmic conversion and piecewise constant approximation," IEEE Trans. Circuits Syst. I, Reg. Papers **69** (2022) 2655 (DOI: 10.1109/TCSI.2022.3167894).
[23] W. Liu, *et al.*: "Design of unsigned approximate hybrid dividers based on restoring array and logarithmic dividers," IEEE Trans. Emerg. Topics Comput. **10** (2020) 339 (DOI: 10.1109/TETC.2020.3022290).
[24] N. Arya, *et al.*: "Energy efficient logarithmic-based approximate divider for ASIC and FPGA-based implementations," Microprocess Microsyst. **90** (2022) 104498 (DOI: 10.1016/j.micpro.2022.104498).
[25] J. Seo and Y. Kim: "High accuracy approximate restoring divider," ICCE-Asia (2021) 1 (DOI: 10.1109/ICCE-Asia53811.2021.9641911).
[26] J. Jeong and Y. Kim: "ASAD-RD: accuracy scalable approximate divider based on restoring division for energy efficiency," Electronics **10** (2021) 31 (DOI: 10.3390/electronics10010031).
[27] K.J.N.S. Bhargav, *et al.*: "A newton raphson method based approximate divider design for color quantization application," ISOCC (2021) 115 (DOI: 10.1109/ISOCC53507.2021.9613961).
[28] N. Arya, *et al.*: "READ: a fixed restoring array based accuracy-configurable approximate divider for energy efficiency," Integration **76** (2021) 1 (DOI: 10.1016/j.vlsi.2020.08.002).
[29] V. Guidotti, *et al.*: "Power-efficient approximate Newton–Raphson integer divider applied to NLMS adaptive filter for high-quality interference cancelling," Circuits, Syst. Signal Process. **39** (2020) 5729 (DOI: 10.1007/s00034-020-01431-9).
[30] N. Arya, *et al.*: "Bit significance based reconfigurable approximate restoring dividers and square rooters," Microelectronics J. **104** (2020) 104861 (DOI: 10.1016/j.mejo.2020.104861).